

S04 & S09 MCO1 – Convex Hull (Application of Stack Data Structure & Sorting Algorithms)

Groupings : Minimum of 2, maximum of 3 members in a group (sign-up in Canvas Group)
Deadline : Refer to the Canvas Submission Page
Percentage : 20% of the Final Grade
Programming Language: C Language

I. HONESTY POLICY, RESPONSIBLE AND ETHICAL USE OF AI

On Honor Code and Honesty Policy: You are encouraged to read and gather information you need to implement the project. **However, please take note that you are NOT allowed to copy-and-paste AS IS 100% from any existing related program code from the internet or other sources (such as printed materials like books that are not online).**

On Responsible and Ethical Use of AI: Please note that the project is meant to serve as a learning experience to further develop your analytical and critical thinking skills. **Your first instinct therefore should be to UTILIZE and MAXIMIZE the COLLECTIVE BRAIN POWER of the members of your group, and not to rely on AI at the onset!** You may use AI as a supporting tool on a **LIMITED BASIS** which means AI should not be a total replacement for you – human! You take **FULL RESPONSIBILITY** in checking and verifying AI generated contents. As part of the project documentation (see accompanying template **Documentation-Template.docx**), you are required to declare, document and articulate the degree of AI use in your project as follows:

- **NONE** - No AI was used for a certain task
- **SCAFFOLDING** - AI was used to build the foundation for a certain task, e.g., brainstorming, solution ideation
- **REFINEMENT** - AI was used to review, provide feedback, and revise outputs for a certain task
- **GENERATION** - AI was used to generate portions of the output for a certain task

When in doubt (if the nature of your AI use is allowed or not), please post your concern in our Canvas MCO Q&A thread.

Gentle reminder: as per Student Handbook (5.2.4.2), *“faculty members have the right to demand the presentation of a student’s ID, to give a grade of 0.0, and to deny admission to class of any student caught cheating under Sec. 5.3.1.1 to Sec. 5.3.1.1.6. The student should immediately be informed of his/her grade and barred from further attending his/her classes.”*

II. WORKING WITH GROUPMATES

Form a group of at least 2 to at most 3 members. The group may be composed of students from different sections (handled by the same teacher). Sign-up in the Canvas MP Groups. **Make sure that each member of the group has approximately the same amount of contribution for the project.** Problems with groupmates must be discussed internally within the group, and if needed, with the teacher.

III. PROBLEM DESCRIPTION

Let’s apply what you learned on stack data structure and sorting algorithms.

For this project, you will implement a program that will compute the **convex hull** of a given set of 2D points using the **Graham’s Scan algorithm**. The original algorithm was developed by Ronald L. Graham, and it was published in "An Efficient Algorithm for Determining the Convex Hull of a Finite Planar Set," *Information Processing Letters*, 1(4), 1972 pp. 132–133. doi:10.1016/0020-0190(72)90045-2).

Please see also the following online resources:

- <https://youtu.be/B2AJoSzf4M?si=X2BOmHcTCRJYNcm->
- <https://visualgo.net/en/convexhull>
- [Graham scan - Wikipedia](#)

There are several implementations of said Graham’s scan algorithm that you can find on the internet, for example,

- [Convex hull - Rosetta Code](#)
- [Convex Hull using Graham Scan - GeeksforGeeks](#)

There is no problem with me if you read, study and learn from these publicly available codes. **However, please take note that for this MCO, you are NOT allowed to copy and use a Graham’s Scan algorithm source code AS IS from some source. Non-compliance will result in cheating punishable with a final grade of 0.0 for this course.** You’ll have to implement the algorithm (i.e., with your groupmates) using the C programming language as specified in the list of **TASKS** described in **Section VI** below.

IV. INPUT

The input is a text file containing the following data: the 1st line encodes n which represents the input/problem size, i.e., the number of points. Thereafter, it is followed by n lines of data where each line contains two **double** data type values representing the x and y coordinates of the points. One or more blank spaces separate the x and y coordinates. The coordinates are NOT sorted in any manner.

Refer to the accompanying example input text files named

- **sample-input.txt** (which contains collinear points) and
- **input-circle.txt** (where all points are actually points on a circle that has a radius of 2, and whose centroid is at the origin 0, 0.)

Use only a simple **fscanf()** to input the text file contents. For example,

```
fscanf(fp, "%d", &n); // fp is the file pointer, n is the number of points
for (int i = 0; i < n; i++) {
    fscanf(fp, "%lf %lf", &X[i], &Y[i]);
    // X and Y are 1D array of double data type values
    // You are free to use different variable name and data structure for the points
}
```

STRICT REQUIREMENT #1: make sure that your program will be able to store and process a maximum problem size of $n = 2^{15} = 32768$ elements.

V. OUTPUT

The output is another text file containing data about the convex hull. The 1st line encodes m which is the number of points in the convex hull. Thereafter, it is followed by m lines where each line contains two **double** data type values representing the x and y coordinates of the points making up the convex hull. One or more blank spaces separate the x and y coordinates.

STRICT REQUIREMENT #2:

- The x, y coordinate values should be formatted such that there are SIX digits after the decimal point.**
- The points should be listed in the output text file such that the first is the anchor, followed by the succeeding points such that when the points are connected, they trace the convex hull in counterclockwise direction.**

Study the accompanying example output text files **sample-output.txt** and **output-circle.txt**.

VI. YOUR TASKS

Task 1. Understand the project specifications (problem) & assign (approximately) EQUAL WEIGHT responsibilities.

- Each member should CAREFULLY read and understand this project specifications document. Thereafter, have a group discussion to ensure that all members of the group have the same base understanding. Agree with group members who are the person RESPONSIBLE for each of the different tasks to accomplish the project.

Task 2. Design, implement and test your stack data structure module.

- Implement a stack data structure with the following operations (1) **CREATE(S)**, (2) **PUSH(S, elem)**, (3) **elem = POP(S)**, (4) **elem = TOP(S)**, (5) **ISFULL(S)** and (6) **ISEMPTY(S)**.
- **VERY IMPORTANT: Implement as an additional stack operation **elem = NEXT-TO-TOP(S)** which will return the element below the top element.** For example: create a stack S . Then push 10, then push 20, and then push 30. **TOP(S)** will return 30. **NEXT-TO-TOP(S)** will return 20.
- You have to decide on your own if you would like to represent the stack using an array or a linked list.
- **STRICT REQUIREMENT #3: make sure that your stack can store up to a maximum of $n = 2^{15} = 32768$ elements.**
- Practice modular programming. That is, compartmentalize the data structures and operations by storing the implementation codes in separate source code files. The codes for stack data structure implementation are stored in the source files named as **stack.h** and **stack.c**. Note that there should NOT be a **main()** function inside **stack.c**.
- Create a driver program to test your stack module. The driver program has its own **main()** function which calls all the stack related functions. Name the stack driver program as **test_stack.c**. Compile it and then run the stack driver exe file to test, and check the correctness of your stack data structure implementation.

Task 3. Implement and test your TWO sorting algorithms module.

- Sorting is a crucial step in Graham's Scan algorithm. It is the step that contributes the most to its time complexity (performance).
- For this project, you will explore and compare the performance of a relatively "slow" sorting algorithm versus a "fast" sorting algorithm:

- o “Slow”: one of bubble sort, selection sort, insertion sort (quadratic algorithms)
 - o “Fast”: one of merge sort, heapsort, quicksort (linear log algorithms)
- Refer to the accompanying file for the **MCO1-Sorting-Algorithms-Assignment.PDF** for the actual algorithms that your group will have to use. For example, Group 1 should use selection sort for the slow algorithm and quick sort for the fast algorithm.
- VERY IMPORTANT: sorting is based on the polar angle made by the anchor point P_0 with another point, say P_i . The anchor point is the point with the lowest (minimum y coordinate). If there are several points with the same minimum y coordinate, choose the point with the lower x coordinate. Refer to Figure 1 below. Read also about polar coordinates in [Polar and Cartesian Coordinates \(mathsisfun.com\)](http://mathsisfun.com)**

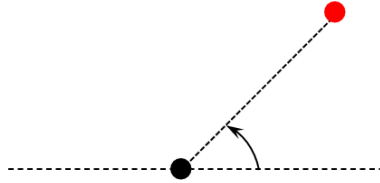


Figure 1: the polar angle is the angle, measured in radians, made by the line segment from the anchor point (black circle) to another point (red circle) with respect to the line parallel to the x-axis.

- Implement the two sorting algorithms. Encode also any helper functions that you need to accomplish the sorting task. Depending on how you implement the Graham Scan algorithm, you may need to implement helper functions for computing the polar angle.
- Practice modular programming. Compartmentalize the two sorting algorithm implementations in the source files **sort.h** and **sort.c**.
- Create a driver program to test your sorting module. The driver program has its own **main()** function which calls the slow sort and then the fast sort algorithm implementations. Name the sort driver program as **test_sort.c**. Compile it and then run the sort driver exe file to test, and check the correctness of your sorting module.

Task 4. Implement two versions of Graham’s Scan algorithm.

- Implement Graham’s Scan algorithm (which should call the functions for sorting, and the necessary stack operations).
- There should be two versions. The first version uses a “slow” sorting algorithm, and the second version uses a “fast” sorting algorithm.
- STRICT REQUIREMENT #4: Both versions should have a `printf()` statement that displays the elapsed time (in milliseconds) between the start and end of the algorithm implementation execution.** Read, understand and learn from the accompanying C program **empirical-time.c** on how to compute the elapsed time using the **clock()** library function.
- Practice modular programming by compartmentalizing your solution into two separate source files named as **graham_slow.c** (for the “slow” version), and **graham_fast.c** (for the “fast” version). Note that there should NOT be a **main()** function inside **graham_slow.c** and **graham_fast.c**.

Task 5. Implement two driver programs to test the Graham Scan’s Algorithm implementation.

- Each driver program should have its own **main()** function that will do the following sequence of steps:
 1. ask the user to input the filename and extension of the input text file (example: MYDATA.TXT)
 2. ask the user to input the filename and extension of the output text file (example: MYHULL.TXT)
 3. open the input text file using **fopen()** library function
 4. read the input text file contents using the **fscanf()** library function
 5. call the appropriate version of the Graham scan algorithm implementation
 6. write the m output points comprising the convex hull into the output text file using **fprintf()** library function
 7. close the input and output text files using the **fclose()** library function.
- Practice modular programming by compartmentalizing your solution in two separate source files named as **test_slow.c** (for the “slow” version) and **test_fast.c** (for the “fast” version).

STRICT REQUIREMENT #5: CODING INSTRUCTIONS (for Tasks 2 to 5):

For all source codes that you will develop: **strictly follow** the coding guidelines/instructions/restrictions in the accompanying templates **stack.h**, **stack.c** and **test_stack.c**.

Task 6. Black box test your Graham’s Scan Algorithm implementation.

- Perform exhaustive testing!
- Create **10** test case text files with different values of n with a minimum of $n = 2^6 = 64$ points. Set the values of n such that they are expressed using 2 as exponent (for example $n = 2^8 = 256$).
- Run the two versions of your program for each test case.
- Tabulate the elapsed time for each test case – this will be used in the **Analysis** section of the project documentation. Discuss the results with your group mates and answer the question “How does the slow and fast version behave as n increases?”.
VERY IMPORTANT: Stress test your codes/solutions! Your 10th (last) test case text file should contain $n = 2^{15} = 32768$ points.

Task 7. Document your project.

- Note that although this task is listed last, you should actually be taking notes and documenting your project starting from Task 1. Don’t forget to jot down notes and document the degree by which you used AI in each task.
- Complete the contents of the accompanying template **Documentation-Template.docx**.

VII. DELIVERABLES

Submit via Canvas a ZIP file named **GROUPNUMBER . ZIP** (for example 01.ZIP for group number 1) which contains:

- Source files (.h and .c files) of the modules as specified above. Make sure to include sufficient and proper code documentation. **Write/encode explicitly the programmer and code tester names in the first few lines of EACH function. Describe what a function does, and briefly describe lines of code that may not be easy to understand.**
- Input text files (.txt) that you used in testing.
- Corresponding output text files (.txt) using the fast version. Actually, the slow version should produce the same output values so there’s no need to submit the slow version output files.
- Documentation named **GROUPNUMBER . PDF** (for example 01.PDF for group number 1).

Do NOT include any EXEcutable file in your submission!

VIII. RUBRIC FOR GRADING

Criteria	RATINGS		
1. Stack	COMPLETE [20 pts] Correctly implemented the stack data structure.	INCOMPLETE [max. 15 pts] Implementation is not entirely correct. Insufficient or incomplete code documentation.	NO MARK [0 pt] Not implemented.
2. Sorting algorithms	COMPLETE [10 pts] Correctly implemented the slow and fast sorting algorithms.	INCOMPLETE [max. 8 pts] Implementation is not entirely correct. Insufficient or incomplete code documentation.	NO MARK [0 pt] Not implemented.
3. Graham’s Scan algorithm	COMPLETE [40 pts] The implementation Graham’s Scan algorithm worked correctly on ALL test data used during the project demo.	INCOMPLETE [max. 30 pts] The implementation Graham’s Scan algorithm worked correctly on some but not all test data used during the project demo. Insufficient or incomplete code documentation.	NO MARK [0 pt] Not implemented.
4. Project Documentation and Analysis	COMPLETE [15 pts] Required details were covered. Point distributions: a. Comparison Table [4 pts] b. Line Graphs [4 pts] c. Analysis [4 pts] d. Remainder of the document [3 pts]	INCOMPLETE [less than 15 pts] Some required details were not included.	NO MARK [0 pt] No documentation.
5. Compliance with Instructions	COMPLIANT [5 pts] All instructions were complied with.	NON-COMPLIANCE Deduction of 1 point for every instruction not properly complied with. Examples: included an EXE file in the ZIP file, file naming not followed.	NO MARK [0 pt] More than 4 instructions not complied with.
6. Project Demo	CONVINCING [10 pts] Was able to answer the questions (asked by the teacher during demo) convincingly, and was able to do a proper project demo.	PARTIALLY CONVINCING [max. 7.5 pts] Was able to answer the questions BUT not all answers were logical or convincing; and/or the demo procedure was not fully followed properly.	NO MARK [0 pt] Unable to do a proper demo and/or cannot answer all the questions.
Maximum Total Points: 100			

Questions? Please post your questions in our Canvas MCO Discussion thread. I will answer MCO related questions ONLY AFTER you post it. Please cooperate. The intent is for all students to be properly informed of all the Questions and my corresponding Answers.

サルパドル・フロランテ